GRADE 11 · TERM 3

Databases: Design, Data & Delphi

A pencil-and-paper workbook: design the tables first, learn to trust the data, then make Delphi walk through the records.

Name: _____

Class: _____ Date: _____

How to use this workbook

1. Work with a **pencil**. Mistakes are how we learn: rub out and try again.
2. Start each unit with **Think First**. Try it on your own before you read the explanation.
3. Read **Learn It**, then do the activities. Trace tables and code are written **by hand**.
4. At the end of each unit, tick the "I can" boxes you are sure of. Write down anything you still want to ask.
5. The **checkpoint test** is done under test conditions, with no notes.

In this workbook

1. Tables on paper
2. Tables that talk to each other
3. Data you can trust
4. The database in Delphi
5. Changing the data
6. Design and screens
7. Term checkpoint test

> **Matches the Annual Teaching Plan:** Term 3 — Databases in Delphi (no SQL): TADOConnection, TADOTable, TDataSource, TDBGrid; First, Next and Eof; Insert, Edit, Delete and Post; searching with loops; multi-form design. Theory: records, fields and keys; relationships and ERDs; referential integrity; data quality and validation; human and computer error; cybercrime.

UNIT 1 Tables on paper

Before there is any Delphi, a database is just tidy paper: rows, columns and a rule for telling one row from another. In this unit you design tables with a pencil.

By the end of this unit I can: (tick each box when you are sure)

- ☐ name the parts of a database: table, record and field
- ☐ choose a sensible field type for a piece of data
- ☐ explain what a primary key is and pick a good one
- ☐ spot a badly designed table
- ☐ design a simple table from a description

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

1.1 Mama Thandi's exercise book

Mama Thandi runs a spaza shop and keeps her stock in an exercise book, one line per item:

Item	Price	How many	Supplier
Bread (brown)	R16	30	Bakers, 021 555 0142
Maize meal 5kg	R54.90	12	Mills, 021 555 0177
bread brown	16.50	a few	Bakers 021 555 0142
Airtime R10	10	lots	—

(a) Which two lines are probably the same item? (b) She wants the total number of items in the shop. Why can she not add up the "How many" column? (c) She wants all items cheaper than R20. Why is the Price column hard to use? (d) Write two rules she could follow so that every line is written the same way.

THINK FIRST · no computer needed

1.2 Who can share?

A school secretary builds a table of learners. For each piece of information, write **YES** if two different learners could have exactly the same value, or **NO** if never. Surname: _____ Grade: _____ Parent's cell number: _____ Date of birth: _____ Admission number (given out by the school): _____.

Which one piece of information could the secretary use to find **one** learner without ever mixing them up? Why?

LEARN IT

Words you must know

A **database** is an organised collection of data. It is stored in one or more **tables**, and each table holds data about **one kind of thing** (learners, items for sale, taxis). Every **row** is a **record**: everything about **one** thing. Every **column** is a **field**: one piece of information, with a name and a type.

`tblItems`

ItemID PK	ItemName	Category	Price	InStock
1	Vetkoek	Food	R8.00	24
2	Pie	Food	R14.00	12
3	Cooldrink	Drinks	R12.00	0
4	Juice	Drinks	R10.50	0

This table has **5 fields** and **4 records** (the heading row is not a record). By convention a table name starts with **tbl** and a field name has no spaces.

LEARN IT

Field types

Each field has a **type** that decides what it may hold. Choosing the right type lets the computer sort, compare and calculate correctly.

Type	Holds	Example
Text	letters, digits, symbols (up to 255)	Surname, cell number
Number (Integer)	whole numbers	InStock = 24
Number (Double)	numbers with decimals	height 1.68 m
Currency	money, fixed decimals	Price = R14.00
Date/Time	dates and times	date of birth
Yes/No	True or False (Boolean)	FeesPaid
AutoNumber	the database gives the next whole number	ItemID
Memo	long text, over 255 characters	a teacher's comment

A cell number is Text, not Number. You never add two cell numbers, and a Number field would drop the leading 0. Rule of thumb: if you will not calculate with it, it is probably Text.

LEARN IT

The primary key

A **primary key** (PK) is a field whose value is **different in every record**, so it picks out exactly one record. A good key is **unique**, **never empty** and **never changes**. Names make poor keys because two learners can share a name, so we add an ID field, often an **AutoNumber**. Sometimes two fields together form the key (for example StudentNo plus SubjectCode); that is a **composite key**.

1.3 Vocabulary [6]

Every row in a table is called a _____ and every column is called a _____. The field that is different in every record, and is never left empty, is the _____. A table with 6 fields and 30 rows of data has _____ records. A cell-number field should be of type _____, because a Number field would drop the leading _____.

1.4 Choose the field type [7]

Write the best field type for each piece of data. Use the types from *Learn It*.

Data	Best field type
The price of a vetkoek (R8.50)	
A learner's date of birth	
Whether the school fees have been paid	
The number of pies in stock	
A parent's cell number	
An admission number that the database hands out by itself	
A teacher's long comment on a learner	

1.5 Pick the primary key [4]

Circle the field that would make the best primary key for each table.

1	tblLearners (Surname, FirstName, Grade, AdmissionNo)	**Surname** **Grade** **AdmissionNo**
2	tblTaxis (Owner, Route, Colour, RegNo, a number plate)	**Colour** **RegNo** **Route**
3	tblBookCopies (Title, Author, Year, BarcodeNo)	**Title** **BarcodeNo** **Author**
4	tblItems (ItemName, Price, ItemID)	**ItemName** **Price** **ItemID**

1.6 Why not the surname? [3]

Tumi suggests using **Surname** as the primary key of tblLearners. Explain in two or three sentences why this is a bad idea.

1.7 Spot the flaws [4]

Here are two records from a badly designed learner table:

Name	Grade	Cell	Joined	Fees
Sipho Dlamini	Eleven	082 123 4567	last Tuesday	R450 paid
Sipho Dlamini	11	0821234567	14/1/2026	450

Find **four** problems. For each one say what is wrong and how a better design would fix it.

1.8 Design a table: the taxi rank marshal [8]

The marshal at a taxi rank records, for every taxi, the data below. Give each a **field name** (no spaces) and a **field type**, and write **PK** next to the field you would use as the primary key.

Data	Field name	Field type	Key?
Number plate			
Driver's name			
Route (for example Bellville to Cape Town)			
Number of seats			
Date the licence expires			
Does it have a roadworthy certificate?			

UNIT 2 Tables that talk to each other

One giant table that repeats everything wastes space and invites mistakes. Good databases split the data into several tables and link them with keys. You will learn the three kinds of link and draw them as diagrams (ERDs).

By the end of this unit I can: (tick each box when you are sure)

- ☐ explain why we split data into several tables
- ☐ name the relationships 1:1, 1:M and M:M and give an example of each
- ☐ say which table holds the foreign key
- ☐ resolve a many-to-many relationship with a linking table
- ☐ draw an ERD for a short scenario
- ☐ explain referential integrity and what an orphan record is

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

2.1 The repeating slips

The tuck shop keeps one slip per sale and copies the learner's name and class teacher onto every slip:

Slip	Learner	Class teacher	Item	Amount
1	Naledi Mokoena	Mr Adams	Pie	R14
2	Naledi Mokoena	Mr Adams	Juice	R10.50
3	Ryan Pillay	Mrs Botha	Chips	R6.50
4	Naledi Mokoena	Mr Adam	Vetkoek	R8
5	Ryan Pillay	Mrs Botha	Water	R9

(a) Spot the typing mistake. (b) Mr Adams retires and Mr Hendricks takes over the class. If the class has 200 slips, how many places must change? (c) Describe a way to store the information so that each teacher's name is written **only once**.

THINK FIRST · no computer needed

2.2 How many on each side?

Write ONE or MANY: how many of the second thing can **one** of the first have, and the other way round?

Pair	One of the first has...	One of the second has...
Learner and school locker (one each)	_____	_____
Class and the learners in it	_____	_____
Learner and the subjects the learner takes	_____	_____
Taxi route and its taxis (each taxi has one route)	_____	_____
Learner and ID number	_____	_____

Write the row numbers that are 1:1, 1:M and M:M.

LEARN IT

One-to-many and the foreign key

Instead of repeating a teacher on every slip, put classes in their own table and store only the class code in each learner's record. That code is a **foreign key** (FK): a field that holds the **primary key of another table** and so links the two. One class has many learners, so this is a **one-to-many (1:M)** relationship. **The foreign key always lives in the table on the MANY side.**

`tblClasses`

ClassID PK	Teacher
11A	Mr Adams
11B	Mrs Botha

`tblLearners`

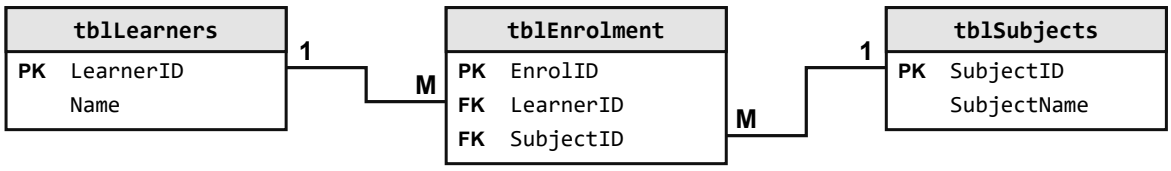
LearnerID PK	Name	ClassID FK
101	Naledi	11A
102	Ryan	11B
103	Zinhle	11A

When Mr Adams retires, only **one** record changes.

LEARN IT

One-to-one and many-to-many

In a **one-to-one (1:1)** link each record matches exactly one record in the other table (learner and locker). It is uncommon, and the foreign key can go in either table. A **many-to-many (M:M)** link, such as learners and subjects, cannot be stored with a single foreign key. The fix is a **linking table** that holds one foreign key for each side. Every M:M becomes two 1:M links.



LEARN IT

Drawing an ERD, and referential integrity

An **entity relationship diagram (ERD)** shows the tables and how they are linked. Draw one box per table with the **table name** on top and the fields below, marking each **PK** and **FK**. Join the boxes with a line from the key field to the foreign key field, with **1** at the one end and **M** at the many end. Steps: (1) list the tables; (2) decide each relationship; (3) replace every M:M with a linking table; (4) put each FK on the many side; (5) draw.

Referential integrity means every foreign key value must match a primary key value that really exists in the parent table. A learner whose ClassID is 11Z when there is no class 11Z is an **orphan record**. The database enforces the rule by refusing to save an orphan, and by refusing (or cascading) the deletion of a class that still has learners.

2.3 Which relationship? [6]

Circle the relationship for each pair.

1	A learner and the one locker allocated to that learner	**1:1** **1:M** **M:M**
2	A school and the learners who attend it	**1:1** **1:M** **M:M**
3	Library members and books (members borrow many books over time; a book is borrowed by many members)	**1:1** **1:M** **M:M**
4	A taxi owner and the taxis he owns	**1:1** **1:M** **M:M**
5	Teachers and the classes they teach (a teacher teaches many classes; a class has many teachers)	**1:1** **1:M** **M:M**
6	A learner and the learner's passport number	**1:1** **1:M** **M:M**

2.4 Where does the foreign key live? [8]

Each line describes a one-to-many link. Write the table that holds the foreign key and a sensible name for the foreign key field.

Relationship	Table with the FK	FK field
One class has many learners (tblClasses, tblLearners)		
One route has many taxis (tblRoutes, tblTaxis)		
One parent has many children at the school (tblParents, tblLearners)		
One doctor has many patients at a clinic (tblDoctors, tblPatients)		

2.5 Draw an ERD: the school library [8]

A school library lends books to members. A member borrows many books over time, and a book is borrowed by many members over time. Each loan records the date the book left the library. Draw an ERD with **three tables**: tblMembers (MemberID, Name, Cell), tblBooks (BookID, Title, Author) and a linking table tblLoans. Show all fields, mark every PK and FK, and write **1** and **M** on the lines.

2.6 Draw an ERD: the taxi association [7]

Every taxi is owned by one owner, but an owner may own several taxis. Every taxi works one route, but a route is served by many taxis. Draw an ERD with tblOwners, tblTaxis and tblRoutes. Give each table a primary key and at least two other fields, mark the foreign keys, and label the lines.

2.7 Explain it [4]

Learners 2 and 4 in tblLearners have ClassIDs 9B and 8C, but tblClasses holds only 8A, 8B and 9A. In three or four sentences explain what is wrong, what referential integrity means, and what the database should do if someone tries to delete class 9A while learners still belong to it. Use the word **orphan**.

UNIT 3 Data you can trust

A database is only as good as the data inside it. You will learn what makes data good, how a program can refuse bad data before it is saved, and what happens when data goes wrong through mistakes or crime.

By the end of this unit I can: (tick each box when you are sure)

- ☐ name and explain the four data-quality characteristics
- ☐ choose the right validation check for a situation
- ☐ calculate and check a check digit by hand
- ☐ explain the difference between validation and verification
- ☐ describe the effects of human and computer error on data
- ☐ explain the effects of identity theft, ransomware and SQL injection in full sentences

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

3.1 Spot what is wrong

Here is part of a class register typed by a tired clerk:

Name	Grade	Mark	Cell	Address
Lwazi Khumalo	11	105	082 555 0191	12 Protea St
Megan Jacobs	Eleven	64	083 555 0127	4 Oak Ave
	11	58	084 555 0163	9 Vine Rd
Pieter Smit	11	71	0845550	31 Rose St
Aisha Patel	11	83	072 555 0188	Flat 4, Kuils River

For each of the five learners write what is wrong and the rule a program could use to stop it. (Aisha moved to Durbanville in January; the register is still her old address.)

THINK FIRST · no computer needed

3.2 A check that adds up

Shops put an extra digit on the end of a code. **The rule:** add up the digits, then choose the extra digit (0 to 9) that makes the total a multiple of 10. For example 5 2 1 adds to 8, so the extra digit is 2 (8 + 2 = 10).
Find the extra digit for 4 1 9 5: ____ for 6 6 3 7: ____ for 2 8 8 0: ____.
A cashier types 4 1 9 5 1 as **4 9 1 5 1** (two digits swapped). Does the rule notice? Why is that a weakness?

LEARN IT

Four questions about data quality

Data has **quality** when it is fit for its purpose. Ask four questions:

Characteristic	The question	Bad data looks like
Accuracy	Is it the right value, free from errors?	a mark of 65 typed as 56
Correctness	Does it follow the rules and allowed values?	Grade holds “Eleven”
Currency	Is it up to date?	the address from before the family moved
Completeness	Is everything that is needed there?	no guardian name on 30 records

A mark of 105 is **incorrect** (it breaks the rule). A mark typed as 56 instead of 65 is **inaccurate** yet looks fine: it obeys every rule, and only checking against the source finds it. *Garbage in, garbage out*: wrong data produces wrong reports and wrong decisions.

LEARN IT

Validation and verification

Validation is the program checking that data is sensible **before** it is saved. It cannot tell whether the data is *true*, only whether it is acceptable. **Verification** checks that data was *copied* correctly: typing it twice, or proofreading against the source.

Check	What it tests	Example
Presence	something was entered	surname not blank
Type	right kind of data	Age holds digits only
Range	between a minimum and maximum	mark 0 to 100
Format	right pattern	e-mail has an @ and a full stop
Lookup	value is in a list	one of the 9 provinces
Check digit	a calculated digit still matches	ID number, barcode

Delphi

```
if edtName.Text = '' then   // presence
  ShowMessage('Please enter a name')
else if (StrToInt(edtMark.Text) < 0) or (StrToInt(edtMark.Text) > 100) then   // range
  ShowMessage('The mark must be from 0 to 100')
else if Pos('@', edtEmail.Text) = 0 then   // format
  ShowMessage('The e-mail address needs an @');
```

LEARN IT

How a check digit works

Our scheme: give the four digits the **weights 1, 2, 3, 4**, multiply, add, and take the **remainder after dividing by 10** (MOD 10). That remainder is the check digit, written as the fifth digit. To check a code, recalculate from the first four digits and compare with the fifth.

Digit	4	1	9	5
Weight	1	2	3	4
Digit × weight	4	2	27	20

Total = 4 + 2 + 27 + 20 = **53**. 53 MOD 10 = **3**, so the full code is **41953**. Swap two digits: 4 9 1 5 gives 4 + 18 + 3 + 20 = 45, MOD 10 = 5, which no longer matches the 3, so the error is caught. Real bank cards and South African ID numbers use a cleverer cousin of this idea, the *Luhn* method.

LEARN IT

When data goes wrong: mistakes and crime

Human error is a mistake by a person: typing 4 120 instead of 1 420, swapping digits, misreading handwriting, deleting the wrong file, or forgetting to update a record. **Computer error** comes from the machine or the program: a power failure (such as load shedding) in the middle of saving, a failing disk, a bug in a formula, or a corrupted file. Either way the data is wrong or missing and so are the decisions based on it. Validation, backups and a UPS reduce the risk.

Identity theft: a criminal uses stolen personal details (ID number, bank details) to open accounts or borrow money in someone else's name; the victim may be refused credit and spend months clearing their name. **Ransomware** locks or encrypts an organisation's files and demands payment to release them; the business or hospital cannot work, and without a recent backup the data may be lost for good. **SQL injection** is an attack on a program that sends what the user types straight to its database: the attacker types cleverly written text into a form so that the database obeys it, and can read, change or delete records. Programmers defend against it by validating every input and never mixing user text into database commands. In South Africa the **POPIA** law requires organisations to protect personal information.

LEARN IT

How to answer a theory question

Write theory answers in **sentences**, not bullet points. Here is why: in a marking guideline a list of bullets is credited only up to the number of points asked for (the first few bullets), so a long list wastes your time. A marker must read a **whole paragraph**, and a good one earns a mark for each separate, well-explained point. Use the shape **point → explain → effect or example**: "Ransomware locks the clinic's files *so* the nurses cannot see patient records, *which* delays treatment."

3.3 Which quality problem? [4]

Circle the data-quality characteristic that has been broken.

1	A parent changed jobs last year but the school still has the old work number.	**Accuracy** **Currency** **Completeness**
2	A mark was typed as 56 but the test paper says 65.	**Accuracy** **Correctness** **Currency**
3	The Grade field holds "Eleven" although only the numbers 8 to 12 are allowed.	**Correctness** **Currency** **Completeness**
4	Thirty learners have no guardian name recorded.	**Accuracy** **Completeness** **Currency**

3.4 Which validation check? [6]

Circle the check that matches each rule.

1	A learner's mark must be from 0 to 100.	**Range** **Type** **Format**
2	An e-mail address must contain an @ and a full stop.	**Format** **Presence** **Lookup**
3	The surname field may not be left empty.	**Presence** **Range** **Check digit**
4	The province must be one of the nine names in a list.	**Lookup** **Format** **Check digit**
5	The last digit of a barcode is recalculated and compared.	**Check digit** **Range** **Type**

3.5 Calculate and check [8]

Use the scheme from *Learn It* (weights 1, 2, 3, 4; MOD 10). For each five-digit code, work out the weighted sum of the **first four digits**, the check digit it should have, and decide whether the code is valid. Look at the first two codes: they use the same digits.

Full code	Weighted sum	Check digit it should have	Valid?
26086			
62086			
73512			
90425			

3.6 Write the function [8]

Write a function **IsValidCode** that receives a five-digit code as a String (such as '41953') and returns True if the fifth digit equals the check digit calculated from the first four (weights 1, 2, 3, 4; MOD 10). Hint: *Ord(sCode[i]) - Ord('0')* turns the character at position i into its digit.

3.7 Validation or verification? [4]

In two or three sentences explain the difference between validation and verification. Then say whether a range check (0 to 100) would notice that a mark of 65 was typed as 56, and why.

3.8 Human error and computer error [5]

During load shedding the power in the school tuck shop fails while the till is saving the day's sales, and the file is damaged. The next morning a helper types the day's takings as R4 120 when the correct amount was R1 420. In one short paragraph name the **computer error** and the **human error**, explain what effect they could have on the tuck shop owner, and say how each could be reduced.

3.9 Effects of cybercrime [11]

(a) A hacker steals the ID numbers and cell numbers of 800 learners from a school database. Explain how this could harm a learner and what the school should do to protect such data. (b) A clinic's patient database is hit by ransomware. Explain what happens and why regular backups matter. (c) In one or two sentences say what SQL injection is and how a programmer can defend against it.

UNIT 4 The database in Delphi

Now the tables come alive on screen. You will learn how four components pass the data from a file to a grid, and how a loop with First, Next and Eof visits every record.

By the end of this unit I can: (tick each box when you are sure)

- ☐ name the components that connect Delphi to a database, in order
- ☐ connect to a database by its path in code
- ☐ move through the records with First, Next and Eof
- ☐ read a field value from the current record
- ☐ trace a loop that visits every record
- ☐ write loops that count, total and find the biggest value

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

4.1 The finger on the table

Work with a partner. One of you is the **table**: the table below is covered, except for the one row under a finger. The other is the **program** and may give only these commands: **FIRST** (finger on the top row), **NEXT** (finger down one row) and **READ** (the table says the values in that row). The program may write on scrap paper.

`tblItems`

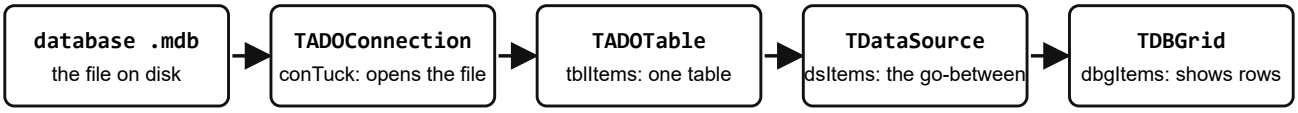
ItemID PK	ItemName	Category	Price	InStock
1	Vetkoek	Food	R8.00	24
2	Pie	Food	R14.00	12
3	Cooldrink	Drinks	R12.00	0
4	Juice	Drinks	R10.50	0
5	Chips	Snacks	R6.50	40
6	Water	Drinks	R9.00	18
7	Muffin	Food	R11.00	5
8	Lollipop	Sweets	R1.50	60

Task: find the **dearest item**. (a) How many times did you use NEXT? (b) How did the program know that there were no more rows? (c) What did the program have to remember on its scrap paper?

LEARN IT

The chain of components

Delphi does not read the file directly. Four components are joined in a chain, and the data flows from the file on disk to the screen:



The first three non-visual components (and the DataSource) sit on a **data module**, here **dmTuck**. The **DBGrid** sits on the form. Every form that uses the data module must have its unit in the **uses** clause. A data-aware control such as a DBGrid can never connect straight to a table: it must go through a DataSource.

LEARN IT

Connecting by path in code

You can build the chain in code, so the path of the database is worked out while the program runs. **ExtractFilePath(Application.ExeName)** gives the folder of the program, so the .mdb file simply sits beside it. Add **ADODB** and **DB** to the uses clause.

Delphi

```
procedure TdmTuck.DataModuleCreate(Sender: TObject);
begin
  conTuck := TADOConnection.Create(Self);
  conTuck.LoginPrompt := False;   // no password box
  conTuck.ConnectionString := 'Provider=Microsoft.Jet.OLEDB.4.0;Data Source='
    + ExtractFilePath(Application.ExeName) + 'Tuck.mdb;';
  conTuck.Connected := True;
  tblItems := TADOTable.Create(Self);
  tblItems.Connection := conTuck;
  tblItems.TableName := 'tblItems';
  tblItems.Active := True;
  dsItems := TDataSource.Create(Self);
  dsItems.DataSet := tblItems;
end;

// on the main form (OnShow, not OnCreate)
dbgItems.DataSource := dmTuck.dsItems;
```

4.2 The components [7]

The _____ opens the database file. The _____ represents one table. The _____ links the table to the grid. The _____ shows the records on the form. The non-visual components live on a _____. The method that moves the pointer to the first record is _____, and _____ is True once the pointer has passed the last record.

LEARN IT

Walking through the records

The table keeps a **pointer** to the **current record**. **First** moves it to the first record, **Next** moves it down one, and **Eof** becomes True when it has moved past the last record. **dmTuck.tblItems['Price']** reads the Price field of the current record. A loop that visits every record always has the same shape: First, then *while not Eof*, do the work, and **Next**. Forgetting Next makes an endless loop.

Delphi

```
rTotal := 0;  iCount := 0;   // set up BEFORE the loop
dmTuck.tblItems.First;
while not dmTuck.tblItems.Eof do
begin
  rTotal := rTotal + dmTuck.tblItems['Price'];   // read the current record
  Inc(iCount);
  dmTuck.tblItems.Next;   // move on, or the loop never ends
end;
if iCount > 0 then
  lblAvg.Caption := FloatToStrF(rTotal / iCount, ffFixed, 8, 2);
```

4.3 Be the computer: dear items [8]

Trace the loop on the tuck-shop table printed in 4.1 (turn back to it). Write whether the price is above R10 and the values of iCount and iUnits **after** each record.

Delphi

```
iCount := 0;  iUnits := 0;
dmTuck.tblItems.First;
while not dmTuck.tblItems.Eof do
begin
  if dmTuck.tblItems['Price'] > 10 then
  begin
    Inc(iCount);
    iUnits := iUnits + dmTuck.tblItems['InStock'];
  end;
  dmTuck.tblItems.Next;
end;
```

ItemName	Price	Price > 10 ?	iCount	iUnits
Vetkoek	R8.00			
Pie	R14.00			
Cooldrink	R12.00			
Juice	R10.50			
Chips	R6.50			
Water	R9.00			
Muffin	R11.00			
Lollipop	R1.50			

4.4 Predict the output [3]

Using the same table, what appears in **memOut**? (FloatToStrF with ffFixed and 2 decimals shows R12.00 for 12.)

Delphi

```
dmTuck.tblItems.First;
while not dmTuck.tblItems.Eof do
begin
  if dmTuck.tblItems['InStock'] = 0 then
    memOut.Lines.Add(dmTuck.tblItems['ItemName']
      + ' (R' + FloatToStrF(dmTuck.tblItems['Price'], ffFixed, 8, 2) + ')');
  dmTuck.tblItems.Next;
end;
```

4.5 Debug: the stock value [4]

This code should add up the value of the stock (price times number in stock) and display it, but it has **four** errors. Find them and write the corrections.

Delphi

```
var
  rValue : Real;
begin
  rValue := 0;
  while dmTuck.tblItems.Eof do
  begin
    rValue := rValue + dmTuck.tblItems['Price'] * dmTuck.tblItems['InStock'];
  end;
  lblValue.Caption := rValue;
end;
```

4.6 Write it: the dearest item [8]

Write code that finds the item with the **highest price** in tblItems and shows its name and price in **lblDear** (for example *Pie: R14.00*). Declare your variables.

4.7 Write it: connect in code [7]

Complete the **connection and the table** in the data module's OnCreate procedure so that the program opens **Tuck.mdb** in the same folder as the program (the DataSource is done for you).

```delphi
procedure TdmTuck.DataModuleCreate(Sender: TObject);
begin
  // ______ your connection and table code goes here ______
  dsItems := TDataSource.Create(Self);
  dsItems.DataSet := tblItems;
end;
```

UNIT 5 Changing the data

A database is no use if you can only look at it. You will add, change and delete records from code, search for records with a loop, and learn the mistakes that quietly destroy data.

By the end of this unit I can: (tick each box when you are sure)

- ☐ use Insert, Edit and Post to add or change a record
- ☐ explain what each dataset state allows
- ☐ write a loop that searches for a record, with a found flag
- ☐ write a loop that changes every record that matches a condition
- ☐ delete matching records safely, without skipping any
- ☐ trace Edit and Delete loops

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

5.1 The strip game

You have a strip of boxes with numbers: **5 0 0 3 0 7 2 0**. Your finger starts on the first box. The aim is to rub out every **0**. When you rub out a box, everything on its right slides one box left to close the gap. The finger stays where it is, so it is now on the box that slid into the gap.

Version A: if the box is a 0, rub it out and keep the finger there; otherwise move the finger one box right. Do this until the finger passes the end. The strip now reads: _____.

Version B: as version A, but after rubbing out a 0 you *also* move the finger one box to the right. The strip now reads: _____. What went wrong in version B?

LEARN IT

Insert, Edit, Delete and Post

A table is normally in **browse** state: you may look but not touch. **Insert** switches to **insert** state (a blank new record) and **Edit** to **edit** state (the current record may change). Neither saves anything. Think of the change as a note on a sticky note: **Post** copies it into the table and returns to browse, and if you forget Post the change is lost. **Cancel** throws the change away. **Delete** removes the current record at once and cannot be undone, so ask the user first.

Delphi

```
dmTuck.tblItems.Insert;                       // add a new item
dmTuck.tblItems['ItemName'] := edtName.Text;
dmTuck.tblItems['Price'] := StrToFloat(edtPrice.Text);
dmTuck.tblItems.Post;

dmTuck.tblItems.Edit;                         // change the current item
dmTuck.tblItems['Price'] := 15;
dmTuck.tblItems.Post;
```

LEARN IT

Searching and changing with loops

To find a record, loop through the table and test each one. Use a **Boolean flag** so you know afterwards whether it was found; the flag also lets the loop stop early (a **Break** inside the loop does the same). To change **every** record that matches, put the Edit and Post pair **inside** the if, and keep **Next outside** the if so that the loop always moves on.

Delphi

```
bFound := False;
dmTuck.tblItems.First;
while (not dmTuck.tblItems.Eof) and (not bFound) do
begin
  if UpperCase(dmTuck.tblItems['ItemName']) = UpperCase(edtFind.Text) then
    bFound := True
  else
    dmTuck.tblItems.Next;
end;
if bFound then
  lblPrice.Caption := FloatToStrF(dmTuck.tblItems['Price'], ffFixed, 8, 2)
else
  lblPrice.Caption := 'Not found';
```

LEARN IT

Deleting matching records: do not skip

After **Delete**, the record that came after the deleted one slides into the current position, so the pointer is already on an **unchecked** record. If you also call Next you jump over it. Move on **only when you did not delete**. (A shortcut for showing a subset of records is the table's **Filter** and **Filtered** properties, for example *Filter := 'Category = ''Drinks'''*, but loops give you full control.)

Delphi

```
dmTuck.tblItems.First;
while not dmTuck.tblItems.Eof do
begin
  if dmTuck.tblItems['InStock'] < 10 then
    dmTuck.tblItems.Delete   // no semicolon: the else belongs to this if
  else
    dmTuck.tblItems.Next;
end;
```

5.2 States and statements [6]

A table that is open and being looked at is in _____ state. _____ prepares a blank new record. _____ lets the current record change. Neither saves anything until _____ is called. To abandon a change you call _____. After Delete, the pointer is already on the _____ record, so you must not call Next as well.

5.3 Debug: the price rise [3]

This code should add R2 to the price of every item in the Snacks category, but nothing is saved and the program sometimes freezes. Find **three** errors.

Delphi

```
dmTuck.tblItems.First;
while not dmTuck.tblItems.Eof do
begin
  if dmTuck.tblItems['Category'] = 'Snacks' then
  begin
    dmTuck.tblItems['Price'] := dmTuck.tblItems['Price'] + 2;
    dmTuck.tblItems.Next;
  end;
end;
```

5.4 Be the computer: Drinks up [6]

Trace the loop on the tuck-shop table printed in 4.1. Write whether the category is Drinks and the price of each item **after** its turn.

Delphi

```
dmTuck.tblItems.First;
while not dmTuck.tblItems.Eof do
begin
  if dmTuck.tblItems['Category'] = 'Drinks' then
  begin
    dmTuck.tblItems.Edit;
    dmTuck.tblItems['Price'] := dmTuck.tblItems['Price'] + 1;
    dmTuck.tblItems.Post;
  end;
  dmTuck.tblItems.Next;
end;
```

ItemName	Category	Category = 'Drinks' ?	Price after
Vetkoek	Food		
Pie	Food		
Cooldrink	Drinks		
Juice	Drinks		
Chips	Snacks		
Water	Drinks		
Muffin	Food		
Lollipop	Sweets		

5.5 Be the computer: the delete loop [8]

The table starts with the same 8 records as in 4.1. Trace this loop. For each pass write whether InStock is below 10, what the loop does, and how many records are left after the pass. **Careful:** after a Delete the pointer is on the next record.

```delphi
dmTuck.tblItems.First;
while not dmTuck.tblItems.Eof do
  if dmTuck.tblItems['InStock'] < 10 then
    dmTuck.tblItems.Delete
  else
    dmTuck.tblItems.Next;
```

Pass	Record tested	InStock	Below 10?	Action	Records left
1					
2					
3					
4					
5					
6					
7					
8					

5.6 Write it: add a new item [5]

The user types the item name in **edtName**, the price in **edtPrice** and the category in **cmbCategory** when **btnAdd** is clicked. Write the code for the click that adds a new record (InStock starts at 0). Do not validate.

5.7 Write it: search by name **[8]**

Write the code for **btnFind** that searches tblItems for the item named in **edtFind** (ignore capital letters). If found, show its price in **lblPrice**; otherwise show **Not found**. Stop looking as soon as it is found.

5.8 Write it: clear the shelf **[7]**

At the end of term the tuck shop deletes every item that has an InStock of 0 **and** a Category of **Sweets**. Write the loop, and ask the user for confirmation once before it starts (MessageDlg returns mrYes for Yes).

UNIT 6 Design and screens

Real programs have more than one screen. Good programmers plan on paper before they open Delphi. You will break a job into forms, choose components, and plan what happens when each button is pressed.

By the end of this unit I can: (tick each box when you are sure)

- ☐ explain why a program uses several forms
- ☐ sketch forms with named components
- ☐ choose the best component for a type of data
- ☐ plan which event opens or closes which form
- ☐ write the code that opens and closes forms

One thing I want to ask my teacher: _____

THINK FIRST · no computer needed

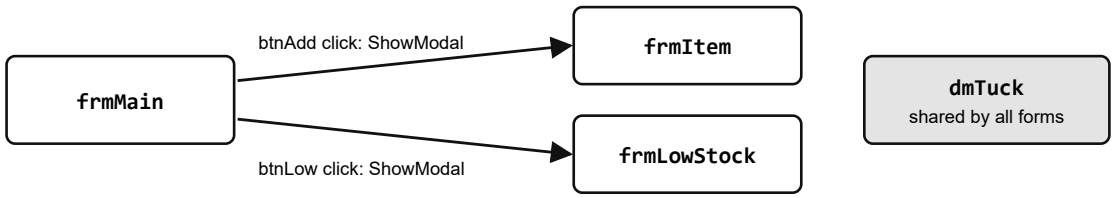
6.1 The tuck-shop manager's wish list

Mrs Naidoo manages the school tuck shop. She wants a program that lets her (1) **see all the items** and search for one, (2) **add a new item**, (3) **change a price or stock level**, and (4) **print a low-stock list**. Decide how many **screens (forms)** she needs and what goes on each. Give each form a name that starts with **frm**. Why not put everything on one form?

LEARN IT

Several forms, one data module

Every form is its own unit. All forms share the **one data module**, so the table is opened once. The main form opens another form with **ShowModal** (the user must finish with the new form first), and a form closes itself with **Close**. A form can only open another if the other's unit is in its **uses** clause (File → Use Unit).



Delphi

```
frmItem.ShowModal;   // in frmMain: the add form opens on top
Close;               // in frmItem: back to frmMain
```

LEARN IT

Choosing components and laying out a screen

Choose the component that makes a mistake **hard to make**. A fixed list of choices (category, province) belongs in a **ComboBox** or **RadioGroup**; a number in a range in a **SpinEdit**; a date in a **DateTimePicker**; a Yes/No in a **CheckBox**; free text in an **Edit** or, if long, a **Memo**. Prefix every name with its type: btnSave, edtName, cmbCategory, sedStock, lblTotal, dbgItems. Good layout: related controls grouped in a panel, labels beside the boxes, the most common job obvious, Tab moving in a sensible order, and clear friendly messages.

6.2 Which component? [6]

Write the best component for each field on the Add Item screen. Use names such as Edit, ComboBox, SpinEdit, CheckBox, DateTimePicker, Memo, RadioGroup.

The user enters…	Best component
The item name (any text)	
The category: Food, Drinks, Snacks or Sweets only	
The number in stock (0 to 500)	
Whether the item is on special	
The date the item was last delivered	
A long note about the supplier	

6.3 Sketch the screens [8]

Sketch two forms for the tuck-shop program in the box below. **frmMain** must show all the items and let the user search and open the Add screen. **frmItem** must let the user enter the name, category and price and save or cancel. Label every component with a proper name (for example btnAdd) and draw an arrow, labelled with the event, from the button that opens frmItem. Then swap with a partner who "presses" your buttons: can they find their way?

6.4 Write it: the event code [8]

Complete the code. btnAdd on frmMain opens frmItem. On frmItem, btnSave must check that the name is not empty (otherwise show a message and stay on the form); if it is valid it inserts the record with the name from edtName and the price from edtPrice, posts it and closes the form.

6.5 Debug: three symptoms [3]

Three things go wrong in a two-form program. For each symptom give the most likely cause and how to fix it.

1. The compiler says *Undeclared identifier: 'frmItem'* inside frmMain's btnAdd code.
2. After pressing Cancel on frmItem, a half-empty record appears in the grid.
3. frmMain opens frmItem with *frmItem.Show*, and the user can click on frmMain while frmItem is open and start a second add.

6.6 Plan the event flow [4]

Write the single line of Delphi (or the two or three lines) that each event must run.

Event	The Delphi code that must run (one to three lines)
btnAdd on frmMain	
btnSave on frmItem (data valid)	
btnCancel on frmItem	
btnQuit on frmMain	

TERM CHECKPOINT

Databases checkpoint

Time: 90 minutes **Total:** 67 marks **Mark:** _____ / 67

No notes, no computer. Show all your working. Write Delphi code neatly and use correct indentation.

SECTION A — Tables, relationships and ERDs [18]

A1 [5]

A school sports database stores matches. Give the best field type for each piece of data.

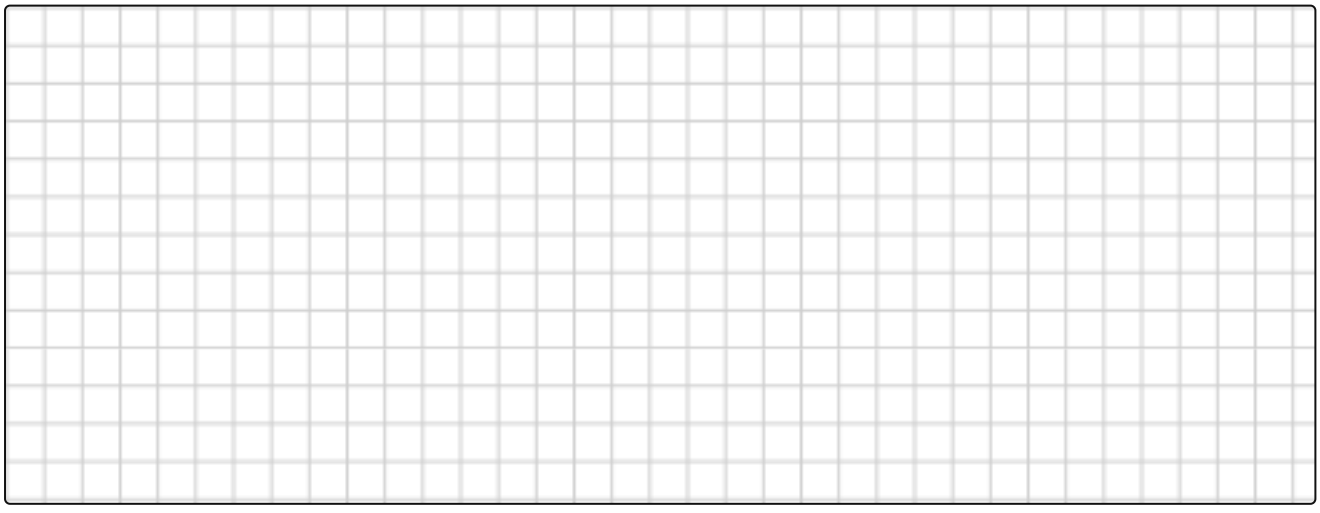
Data	Field type
The date of the match	
The name of the opposing school	
The final score of our team	
Whether the match was played at home	
A match number handed out automatically	

A2 [4]

In tblPlayers (PlayerNo, Surname, TeamID) the primary key is _____ and the foreign key is _____. The foreign key must match a value in the _____ table. A player whose TeamID does not exist there is called an _____.

A3 [9]

Learners may play several sports and every sport has many learners. Each sport has one coach, but a coach may coach several sports. Draw an ERD with **four tables**: tblLearners, tblSignups (a linking table), tblSports and tblCoaches. Give every table a primary key and at least one other field, mark the foreign keys, and write 1 and M on each line.



SECTION B — Databases in Delphi [29]

B1 [8]

The table below is tblLearners. Trace the loop. Write whether the learner is in Grade 11 and the values of iTotal and iPass **after** each record.

tblLearners

Name	Grade	Mark
Aphiwe	10	62
Bianca	11	48
Farai	10	50
Chloe	11	75
Dumisani	12	90
Esihle	11	95

```Delphi
iTotal := 0;  iPass := 0;
dmSchool.tblLearners.First;
while not dmSchool.tblLearners.Eof do
begin
  if dmSchool.tblLearners['Grade'] = 11 then
  begin
    iTotal := iTotal + dmSchool.tblLearners['Mark'];
    if dmSchool.tblLearners['Mark'] >= 50 then
      Inc(iPass);
  end;
  dmSchool.tblLearners.Next;
end;
```

Name	Grade	Grade 11?	iTotal	iPass
Aphiwe	10			
Bianca	11			
Farai	10			
Chloe	11			
Dumisani	12			
Esihle	11			

B2 [3]

This code should add a new learner when btnSave is clicked. Find and correct **three** errors.

```Delphi
dmSchool.tblLearners['Name'] := edtName.Text;
dmSchool.tblLearners['Grade'] := sedGrade.Value;
dmSchool.tblLearners['Mark'] := edtMark.Text;
```

B3 **[4]**

The loop below is meant to delete every learner with a mark under 60, but it has a mistake. Which learners remain, in order? Which learner was wrongly kept, and why?

Delphi

```
dmSchool.tblLearners.First;
while not dmSchool.tblLearners.Eof do
  if dmSchool.tblLearners['Mark'] < 60 then
  begin
    dmSchool.tblLearners.Delete;
    dmSchool.tblLearners.Next;
  end
  else
    dmSchool.tblLearners.Next;
```

B4 **[7]**

Write code that adds **5 marks** to every Grade 11 learner in tblLearners, but never lets a mark go above 100. Save every change.

B5 **[7]**

The user types a learner's name in **edtFind**. Write code that searches tblLearners and shows the learner's mark in **lblMark**, or **Not found** if there is no such learner.

SECTION C — Data, errors and cybercrime [20]

C1 **[4]**

Circle the data-quality characteristic that is broken.

1	A record still shows the address the family left last year.	**Accuracy** **Currency** **Completeness**
2	A mark of 65 was captured as 56.	**Accuracy** **Correctness** **Currency**
3	A required cell-number field was left blank on 40 records.	**Correctness** **Currency** **Completeness**
4	The Grade field contains 15, but only 8 to 12 are allowed.	**Accuracy** **Correctness** **Completeness**

C2 **[4]**

A stock code has four digits and a check digit calculated with weights 1, 2, 3, 4 and MOD 10. (a) Calculate the check digit for **8 1 4 6**. (b) Is the code **58403** valid? Show your working.

C3 **[4]**

Explain in a short paragraph what the difference is between a **human error** and a **computer error**, give one example of each, and state one effect such errors may have on the data in a database.

C4 **[5]**

A hospital's patient database is infected with **ransomware**. In a paragraph of about four sentences explain the effect on the hospital and its patients and how the hospital could have protected itself.

C5 **[3]**

Explain what **SQL injection** is and name one way a programmer can reduce the risk.

COMING UP NEXT TERM · warm-up

What does your gadget know about you?

In Term 4 you will study the **Internet of Things**: everyday objects with sensors that send data over the Internet. Before the theory, think about two gadgets.

A **smart electricity meter** sends the household's usage to the municipality every few minutes. A **fitness band** records steps, heart rate and sleep, and sends them to the maker's servers.
For each gadget write what data it collects, who benefits from it, and one way the data could be misused if it fell into the wrong hands.